

ARCHITECTURE

VisionEngine Architecture

Overview

Go module (`digital.vasic.visionengine`) providing computer vision and LLM Vision for UI analysis, screenshot comparison, and navigation graph building. Used by HelixQA for autonomous QA testing.

Package Structure

```
pkg/  
  analyzer/  -- Core types (ScreenAnalysis, UIElement, Action,  
ScreenIdentity)  
  graph/     -- NavigationGraph (directed graph, BFS pathfinding,  
coverage tracking)  
  llmvision/ -- LLM Vision API adapters (OpenAI, Anthropic,  
Gemini, Qwen, Ollama)  
  opencv/    -- OpenCV stubs (real impl behind `//go:build  
vision` tag)  
  config/    -- Configuration from environment variables  
  remote/    -- SSH-based Ollama deployment and VisionSlot pool
```

Vision Provider Abstraction

All vision providers implement the `VisionProvider` interface:

```
type VisionProvider interface {  
  AnalyzeImage(ctx context.Context, image []byte, prompt  
    string) (string, error)  
  CompareImages(ctx context.Context, img1, img2 []byte, prompt  
    string) (string, error)  
  SupportsVision() bool  
  MaxImageSize() int  
  Name() string  
}
```

Five provider implementations: `OpenAIProvider`, `AnthropicProvider`, `GeminiProvider`, `QwenProvider`, `OllamaProvider`. Each validates inputs (empty image, empty prompt, size limits) before making HTTP calls to the respective API.

`FallbackProvider` wraps multiple providers in a chain. If the primary fails, subsequent providers are tried in order. Thread-safe via `sync.RWMutex`.

Navigation Graph

`NavigationGraph` is the most important type -- imported directly by `HelixQA`. It is a directed graph where:

- **Nodes** are `ScreenNode` (screen identity + visited flag + timestamp)
- **Edges** are `Transition` (from/to screen IDs + action taken)

Key operations:

- `AddScreen()` / `AddTransition()` -- build the graph during exploration
- `PathTo(targetID)` -- BFS shortest path from current screen to target
- `Coverage()` -- ratio of visited screens to total screens
- `UnvisitedScreens()` -- guides the curiosity phase to unexplored areas
- `Export()` -- serializable `GraphSnapshot` for JSON/Mermaid output

Thread-safe via `sync.RWMutex`. Self-transitions are rejected.

Remote Deployment

The remote package manages Ollama vision models on remote hosts via SSH:

1. **Deployer** -- `EnsureReady(ctx)` performs: install Ollama if missing, start service if stopped, pull model if unavailable. Returns the API endpoint URL.
2. **VisionSlot** -- Dedicated inference slot per platform/device. Each slot serializes vision calls via `sync.Mutex` so one slow inference does not block other platforms.
3. **SlotPool** -- Manages multiple `VisionSlots`, one per device under test.

Screenshot Analysis Pipeline

```
Screenshot ([]byte)
|
VisionProvider.AnalyzeImage(image, prompt)
|
LLM response (JSON string)
|
Parse into ScreenAnalysis
  +-- UIElement[]    (buttons, inputs, links with bounding
boxes)
  +-- TextRegion[]   (detected text with positions)
  +-- VisualIssue[]  (overlap, truncation, contrast problems)
  +-- Action[]       (possible navigation actions)
  |
NavigationGraph.AddScreen() / AddTransition()
```

Build Tags

- **Default** (no tags): OpenCV stubs return errors; LLM providers work normally. Suitable for testing and CI.
- **vision** tag: Full OpenCV/GoCV support for local image processing (SSIM, color analysis, element detection, video frame extraction).

Key Design Decisions

- **Stub pattern** for OpenCV: The module compiles without CGO dependencies by default. OpenCV features are behind build tags.
- **FallbackProvider**: Multi-provider resilience ensures QA sessions continue even if one API is down.
- **Per-slot serialization**: Prevents contention when multiple devices run QA sessions simultaneously against a shared Ollama instance.
- **SSH-based deployment**: No agent software needed on remote GPU hosts -- standard SSH is sufficient.